UNIT - 03

HOSPITAL PATIENT MANAGEMENT SYSTEM (Modified)

Problem Statement:

Develop a hospital patient management system in C++ using a circular doubly linked list, an array and a queue. Patient details (ID, name, age, disease, bed number) are stored in the circular list. Beds are managed using an array. Discharged patients are held in a queue, and the oldest can be re-admitted. The system supports admitting, discharging, searching, forward and backward display, bed status and history.

Objective:

The objective of this project is to develop a Hospital Patient Management System in C++ using structures, without classes. Patient details such as ID, name, age, disease and allotted bed number are stored in a circular doubly linked list, which allows easy insertion, deletion, searching and traversal in both forward and backward directions. Hospital beds are managed using an array so a free bed can be allotted to each new patient. Discharged patients are stored in a queue, which keeps the discharge history and lets the oldest discharged patient be re-admitted. The project applies circular linked list, array and queue concepts to solve a real-world hospital problem efficiently.

Algorithm:

Step 1: Start. Create an empty circular doubly linked list (head = NULL), an array of 10 free beds and an empty circular queue (size 5).

Step 2: Display the menu: admit, discharge, search, display forward, display backward, bed status, undo discharge, history, exit.

Step 3: Read the choice and call the matching function. Repeat Steps 2 and 3 until choice = 0.

Step 4: Admit: Find the first free bed in the array. If none is free, display "No beds available" and stop.

Step 5: Read patient ID. If the ID already exists in the list, display "Patient ID already exists" and stop.

Step 6: Read name, age and disease. Allot the bed number, mark the bed as occupied and increase count.

Step 7: Create a new node and insert it at the end of the list (tail = head->prev). Link tail->next = new node, new node->prev = tail, new node->next = head and head->prev = new node.

Step 8: Discharge: Read patient ID and search the list from head until the head is reached again. If not found, display "Patient not found" and stop.

Step 9: If the queue is full, remove (dequeue) the oldest record. Then insert (enqueue) the patient record at rear = (rear + 1) % size.

Step 10: Free the patient's bed in the array. Delete the node by linking its previous and next nodes (update head if the head is deleted), then decrease count.

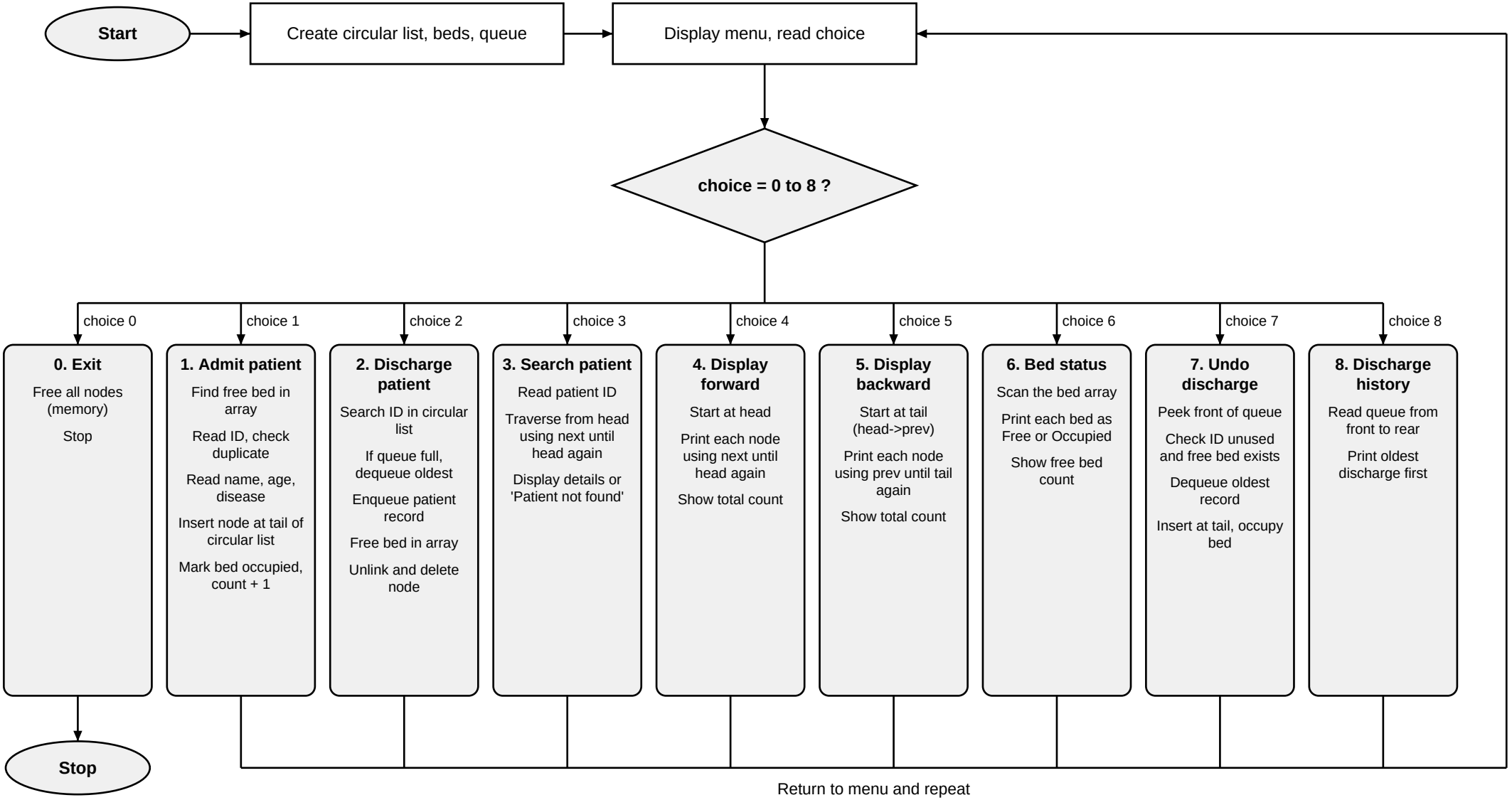
Step 11: Search: Read the patient ID and traverse the list from head using next until head is reached again. If a node matches, display the patient details; otherwise display "Patient not found".

Step 12: Display forward: Start at head and print each node using next until head is reached again. Display backward: Start at tail (head->prev) and print each node using prev until tail is reached again.

Step 13: Bed status: Traverse the bed array and print each bed as free or occupied, along with the free bed count.

Step 14: Undo discharge: If the queue is empty, display a message and stop. Otherwise take the front (oldest) record and check that its ID is unused and that a free bed exists.

Step 15: Dequeue the record (front = (front + 1) % size), allot a free bed and insert the patient at the end of the list.

Step 16: History: Print the queue records from front to rear (oldest discharge first). On exit, free all nodes and Stop.

Flowchart: Hospital Patient Management System